Computer Networks — Assessment-8
Project Proposal

DNScope

DNS Monitoring and Spoofing Detection System

A multi-resolver, history-aware DNS verification dashboard with transparent, explainable alerts

Student name	:	**Anwita Padhi**
Registration number	:	**24BYB0193**
Course	:	**Computer Networks**
Assessment	:	**Assessment-8**
Faculty	:	**Dr C Navaneethan**
Institution	:	**Vellore Institute of Technology (VIT), Vellore**
Academic year	:	**2026–2027**
Project category	:	**Computer Networks and Network Security**

1. Project Title

DNScope — DNS Monitoring and Spoofing Detection System. DNScope is a network measurement and verification tool for the Domain Name System. For any domain the user enters, it sends the same DNS query to several independent recursive resolvers, inspects the returned resource records and header flags, keeps a history of what each domain normally resolves to, and uses transparent rules to flag answers that deserve a closer look. Each flag is shown with the protocol-level evidence that triggered it and the ordinary network reasons that could also explain it. The prototype is planned as a small web application (Python with dnspython for DNS queries, SQLite for storage, and a browser dashboard), but the focus of this proposal is the networking problem, not the software stack.

2. Abstract

The Domain Name System (DNS) is the Internet's distributed directory: almost every TCP or UDP connection begins with a DNS lookup that maps a name to an IP address. Classic DNS runs over UDP port 53 without authentication, so a forged or poisoned response can silently redirect users to an attacker's server. Cache poisoning exploits the fact that a resolver accepts the first response whose 16-bit transaction ID, source port and question match its pending query, and then serves the false record to all its clients until the TTL expires. DNSSEC can prevent this, but only for signed zones and validating resolvers. At the same time, legitimate mechanisms such as round-robin load balancing, content delivery networks (CDNs), geo-DNS, anycast and resolver caching also make answers differ between resolvers and over time, which makes suspicious changes hard to recognise with tools like *dig* or *nslookup*. This project proposes **DNScope**, which queries the system resolver and three public resolvers in parallel, compares their answers, TTLs, response codes and DNSSEC flags with each other and with a per-domain history, and applies six rule-based checks: unexpected address changes, resolver disagreement, abnormal TTLs, repeated resolution failures, DNSSEC validation failures and private addresses in public answers. Every alert is presented as a reason to investigate, not as proof of spoofing. The design will be evaluated with real lookups, legitimate CDN changes and clearly labelled simulated attack responses, measuring precision, recall, false-positive rate and detection latency. No experimental results are reported in this proposal.

3. Literature Review / Related Works

3.1 DNS name space and the resolution process

DNS is a distributed, hierarchical database defined in RFC 1034 and RFC 1035 [1], [2]. The name space is a tree: the unnamed root (.) delegates top-level domains such as *.com* or *.in*, which in turn delegate second-level zones such as *example.com*. Each zone is served by one or more *authoritative* name servers, and delegation is expressed by NS records in the parent zone, with “glue” A/AAAA records when the server's name lies inside the zone it serves.

Fig. 1 shows a typical lookup. The application calls the operating system's *stub resolver*, which sends a **recursive** query (RD = 1, “recursion desired”) to a *recursive resolver* run by the ISP, the organisation or a public provider. If the answer is not cached, the resolver performs **iterative** queries: the root servers refer it to the .com servers, these refer it to the example.com servers, and the authoritative server finally returns the answer. The resolver caches every record it learns for the record's TTL and returns the result to the stub. Because one resolver serves many clients from its

cache, a single false record accepted by the resolver affects every user behind it — which is exactly what cache poisoning exploits.

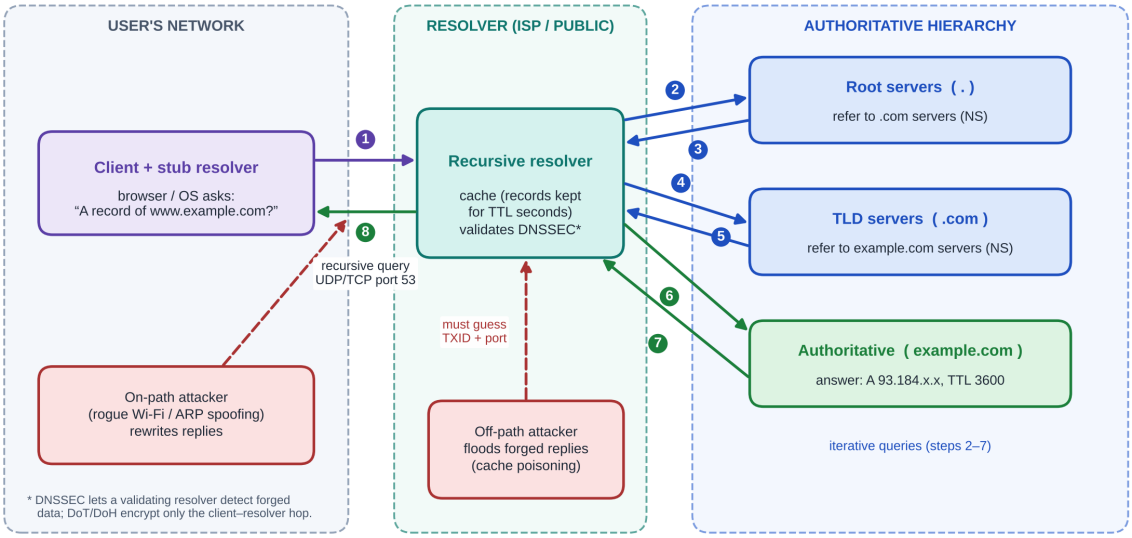


Fig. 1. Recursive and iterative DNS resolution, with the points where an on-path attacker (between client and resolver) and an off-path attacker (racing the authoritative server) can inject forged responses.

3.2 DNS message format

Queries and responses share one binary format [2]: a 12-byte header followed by four sections — *Question*, *Answer*, *Authority* (NS records for referrals) and *Additional* (glue and the EDNS OPT record). Each resource record carries NAME, TYPE (A, AAAA, CNAME, NS, MX, TXT, SOA, ...), CLASS (normally IN), TTL, RDLENGTH and RDATA. Table 1 lists the header fields and why they matter for spoofing detection; DNScope records most of them for every response it receives.

Table 1. DNS header fields (RFC 1035, RFC 4035) and their relevance to this project.

Field	Size	Meaning	Relevance to spoofing and to DNScope
ID	16 bits	Transaction ID chosen by the querier and copied into the response	A forged reply must match it; only 65,536 values, so it is guessable without other entropy
QR, OPCODE	1 + 4 bits	Query (0) or response (1); kind of query (standard = 0)	Used to parse responses
AA	1 bit	Authoritative Answer: sent by a server authoritative for the zone	Recursive resolvers normally return AA = 0; recorded as evidence
TC	1 bit	TrunCated: response did not fit; client should retry over TCP	Triggers TCP retry; TCP is much harder to spoof off-path
RD, RA	1 + 1 bit	Recursion Desired (set by stub) / Recursion Available (set by resolver)	DNScope sends RD = 1 to each resolver
AD, CD	1 + 1 bit	Authenticated Data (resolver validated DNSSEC) / Checking Disabled (skip validation)	Basis of rule R5 (DNSSEC failure detection)
RCODE	4 bits	Result: NOERROR (0), SERVFAIL (2), NXDOMAIN (3), REFUSED (5)	Failure patterns feed rule R4; SERVFAIL can signal a DNSSEC failure
QD/AN/NS/AR COUNT	4 × 16 bits	Number of records in each of the four sections	Used to parse the response; unexpected counts are logged

3.3 Transport, caching and TTL

DNS normally uses **UDP port 53**: one datagram for the query and one for the reply, with no handshake. RFC 1035 limited UDP messages to 512 bytes [2]; the EDNS(0) extension adds an OPT pseudo-record that advertises a larger UDP buffer and carries the **DO** (“DNSSEC OK”) bit that asks for signatures [3]. When a response is too large the server sets TC = 1 and the client repeats the query over **TCP port 53**, which all implementations must support [4]. The lack of a handshake is what makes UDP DNS spoofable: an off-path attacker only needs to send a datagram with the right addresses, ports, transaction ID and question before the genuine reply arrives. TCP's three-way handshake and sequence numbers make the same attack far harder.

Resolvers cache positive answers for their TTL and cache negative answers (NXDOMAIN or “no data”) for a period derived from the zone's SOA record [5]. A cached record is returned with its *remaining* TTL, so two honest resolvers can report different TTLs for the same record. Caching has two consequences for this project: a poisoned record persists for as long as the attacker's chosen TTL, and a check made at the wrong moment may see either the stale genuine answer or the stale false one.

3.4 Why legitimate answers differ

Many large services intentionally return different answers. **Round-robin DNS** rotates through a pool of addresses; **geo-DNS** and CDNs choose an edge server close to the querying resolver, sometimes using the EDNS Client Subnet option that passes part of the client's address to the authoritative server [6]; and the public resolvers themselves are **anycast** services, where one address such as 8.8.8.8 is announced from many sites and BGP routes each client to a nearby instance [7]. Addresses belonging to the same operator usually share a routing prefix, so comparing CIDR prefixes (for example the /24 for IPv4) rather than exact addresses separates “new server in the same network” from “server in an unrelated network” [8]. These effects are the main source of false alarms that any DNS anomaly detector must handle.

3.5 DNS monitoring and packet-analysis tools

Wireshark is a graphical packet analyser that decodes DNS messages field by field from live captures or saved files [9]. It is excellent for learning and for deep troubleshooting, but it shows traffic rather than judgements: the analyst must decide what is abnormal, and it keeps no per-domain memory of earlier answers. **Zeek** is a network security monitor that turns traffic into structured logs; its DNS log records queries, answers, TTLs and response codes for every lookup it sees on a monitored link, and its scripting language allows custom detection logic [10]. **Suricata** is an intrusion detection and prevention engine that applies signature rules, including DNS-specific keywords, to live traffic and produces DNS event logs [11]. Zeek and Suricata are powerful but are designed for passive monitoring at a network vantage point and require sensor deployment, rule management and log analysis. In this proposal these tools are **related work only**; DNScope does not depend on them and does not capture packets.

3.6 DNS spoofing and cache poisoning

RFC 3833 catalogues the main threats to DNS, including packet interception, ID guessing and query prediction, and name-based attacks that inject false data into a resolver's cache [12]. In *cache poisoning*, an off-path attacker races the genuine authoritative server and tries to deliver a forged response that matches the resolver's outstanding query; if accepted, the false record is cached and

served to every client of that resolver until its TTL expires. Son and Shmatikov analysed such attacks systematically and showed how resolver implementation choices affect their feasibility [13]. RFC 5452 standardised defences such as randomising both the 16-bit query ID and the UDP source port to make forged answers harder to match [14]: with only the ID random an attacker must guess one of 65,536 values, while a random source port adds up to roughly 16 more bits, raising the search space towards 2^{32}. Because an attacker can trigger queries for many non-existent subdomains of a target zone, each giving a fresh race, caching alone does not limit the number of attempts [13], [14]. Later work by Man *et al.* demonstrated that side channels can be used to infer the randomised source port, reviving off-path poisoning against widely used resolvers [15]. The lesson for this project is twofold: poisoning remains a realistic threat, and its visible symptom from the outside is often simply a resolver returning a different answer from others.

3.7 DNSSEC and DNS data authenticity

The DNS Security Extensions (DNSSEC) add digital signatures to DNS data so that a validating resolver can check that records really come from the zone owner and were not modified [16]–[18]. Trust is built as a *chain*: the resolver starts from the root zone's key (its trust anchor), checks the DS record that each parent zone publishes for its child, then the child's DNSKEY, and finally the RRSIG signature over the requested records; NSEC/NSEC3 records provide authenticated proof that a name does not exist. RFC 4034 defines these record types [17], and RFC 4035 defines how resolvers validate them and signal the result, including the *Authenticated Data* (AD) bit in responses and the *Checking Disabled* (CD) bit that asks a resolver to skip validation [18]. When validation fails, a validating resolver returns SERVFAIL rather than the forged data. DNSSEC is the strongest available protection against spoofed answers, but it only helps for zones that are signed and for resolvers that validate, and the AD bit is only trustworthy if the path between the client and the resolver is itself trusted [16]. DNScope therefore treats DNSSEC information as useful evidence when it is available, not as a universal answer.

3.8 Encrypted DNS

DNS over TLS (DoT, RFC 7858) and DNS over HTTPS (DoH, RFC 8484) encrypt the channel between a client and its resolver, protecting queries from eavesdropping and on-path tampering on that hop [19], [20]. Encryption does not authenticate the data the resolver itself obtained from authoritative servers, so a poisoned resolver still returns poisoned data over an encrypted channel. Encryption also means that passive tools on the local network can no longer read DNS traffic, which is one reason DNScope uses active lookups instead of packet inspection. dnspython can send queries over UDP, TCP, DoT and DoH, so an encrypted query path can be added without changing the design [21].

3.9 DNS anomaly detection and historical record analysis

Research on malicious-domain detection has shown the value of history. Notos builds a dynamic reputation for domains from passive DNS data, using features such as how the set of IP addresses and networks behind a name evolves over time [22]. EXPOSURE detects malicious domains from large-scale passive DNS traffic using time-based, answer-based and TTL-based features [23]. Both systems rely on large datasets collected at ISP or sensor scale and on machine-learning classifiers, which are beyond the scope of an undergraduate project. They nevertheless motivate two ideas DNScope borrows in a simple form: a domain's past answers form a useful baseline, and TTL behaviour carries signal.

3.10 Comparing answers from independent vantage points

The Perspectives system proposed verifying a server's public key by asking several independent "notary" hosts on different network paths what key they observe, and by looking at how long that key has been seen [24]. The underlying idea, that an attack on one path is unlikely to fool many independent observers at once, applies naturally to DNS: if one resolver's answer disagrees with several others and with history, that is worth examining. It is equally important to recognise the limitation: large services deliberately return different addresses to different resolvers through CDNs and geographic load balancing, so disagreement by itself is weak evidence.

3.11 Comparison of existing approaches

Table 2 compares the approaches above with the proposed project. The final column describes a difference in *scope or implementation*; it is not a claim that DNScope is better than these mature tools at their own purpose.

Table 2. Comparison of existing tools and approaches with the proposed DNScope project.

Approach	Main purpose	Strengths	Limitations for this project	How DNScope differs
Wireshark [9]	Interactive packet capture and protocol decoding	Complete field-level view; excellent for teaching	No judgement or per-domain history; manual analysis; cannot read encrypted DNS	Active lookups with stored history and automatic, explained flags
Zeek [10]	Passive network monitoring and structured logging	Rich DNS logs; scriptable detection	Needs a sensor on the network path and log tooling	Runs on one laptop; no capture; focused on chosen domains
Suricata [11]	Signature-based IDS/IPS	Fast, mature rule engine; DNS keywords	Signatures target known patterns; deployment effort	Behavioural rules on answers over time, not packet signatures
DNSSEC validation [16]–[18]	Cryptographic authenticity of DNS data	Strong guarantee for signed zones	Only for signed zones and validating resolvers	Uses AD bit / SERVFAIL as one rule (R5); covers unsigned zones heuristically
Passive-DNS reputation (Notos, EXPOSURE) [22], [23]	Classify malicious domains at scale	Uses long history and many features	Needs ISP-scale data and ML models	Small per-domain profile and transparent rules instead of ML
Multi-vantage verification (Perspectives) [24]	Detect path-specific attacks by asking independent observers	Simple, intuitive trust signal	Designed for keys, not DNS; legitimate variation exists	Applies the idea to DNS resolvers, combined with history
dig / nslookup	One-off manual DNS queries	Universally available, precise	Single answer, single moment, no memory or interpretation	Same raw data, plus comparison, history and explanations

Summary. Each technique used by DNScope (multi-resolver comparison, historical baselines, TTL analysis, DNSSEC status checks, rule-based alerting) already exists in the literature or in practice. Combining known techniques is not by itself a novel research contribution, and this proposal makes no claim of global novelty or patentability. Its intended value is an understandable, well-explained

integration that a student can build, test and reason about.

4. Problem Statement, Gap Identification & Objectives

4.1 Problem statement

When a user or a small-network administrator suspects that a domain is resolving to the wrong place, the usual response is to run *nslookup* or *dig*. These tools return what one resolver says at one moment. They do not show whether other resolvers agree, whether the answer matches what the domain returned yesterday, or whether a change is normal for that domain. Interpreting the output also requires knowledge many users lack. The core difficulty is that suspicious and legitimate changes look alike:

- **IP addresses** change for routine reasons: hosting migrations, failover, and round-robin load balancing that rotates through a pool of addresses.
- **CDNs and geo-DNS** deliberately give different resolvers different answers depending on their location, so two honest resolvers can disagree completely.
- **TTLs** differ between resolvers because cached answers report the remaining time, and operators lower TTLs before planned changes.
- **Resolver behaviour** differs: some resolvers validate DNSSEC and some do not, some filter malware or adult domains, and some ISPs rewrite NXDOMAIN answers.
- **Encrypted DNS** hides traffic from passive tools on the local network, so packet-based monitoring may not see lookups at all.

A useful tool must therefore collect more context than a single lookup, keep memory across time, and explain its reasoning honestly, including the innocent explanations.

4.2 Research gap and project differentiation

From the review in Section 3, mature tools fall into two groups: low-level analysers that show raw data without interpretation (Wireshark, dig) and sensor-based or data-intensive systems that need network deployment or large datasets (Zeek, Suricata, passive-DNS reputation systems). The **implementation-level gap** this project targets is a lightweight, self-contained tool that brings the following elements together for individually chosen domains:

1. a per-domain historical DNS profile (known addresses, address prefixes, TTL baseline, failure count, DNSSEC status);
2. comparison of the same query across several independent resolvers in a single run;
3. transparent, rule-based detection with configurable thresholds instead of opaque models;
4. a human-readable explanation for every alert, listing the evidence and possible legitimate causes;
5. a unified dashboard for examining observations, comparisons, alerts and domain history.

These are **proposed design contributions**. The individual techniques are not new, and whether the combination offers anything beyond existing commercial monitoring services or research tools would need to be assessed against further literature. The gap is stated at the level of an accessible, explainable implementation, not a research breakthrough.

4.3 Objectives

Table 3. Project objectives and how completion will be measured.

#	Objective	Measurable completion criterion
O1	Retrieve DNS records for user-specified domains	A, AAAA, CNAME, NS, MX and TXT records returned with TTL, RCODE and response time
O2	Compare responses from selected resolvers	Each check queries 4 resolvers (system, 8.8.8.8, 1.1.1.1, 9.9.9.9) and reports agreement per record type
O3	Record domain history	Every observation stored in SQLite; profile updated with known IPs, /24 or /48 prefixes and TTL baseline
O4	Flag suspicious changes with transparent rules	Rules R1–R6 implemented with thresholds read from one configuration file
O5	Explain every alert with evidence	100% of alerts store evidence JSON, rule ID, explanation and possible legitimate causes
O6	Present results in an interactive dashboard	Five pages implemented with loading, empty and error states
O7	Evaluate with normal, unusual and controlled cases	Planned test suite run; precision, recall, false-positive rate and latency reported

4.4 Scope and limitations

In scope. On-demand checks of domains the user enters; four resolvers per check; record types A, AAAA, CNAME, NS, MX and TXT; history stored locally; six detection rules; DNSSEC status taken from validating resolvers where available; a single-user web dashboard running on a laptop; simulated data only in clearly labelled test scenarios.

Out of scope. The first version performs DNS lookups and analyses the responses. It does **not** passively inspect every packet on the user's network, capture traffic, block or correct answers, act as a resolver for other devices, or use machine learning. It runs no distributed sensors and needs no cloud infrastructure.

Limitations. DNScope cannot detect every DNS spoofing attack. In particular: (i) if the attacker controls the authoritative server or the domain registration, every resolver returns the same false answer; (ii) poisoning that affects only the victim's own resolver is visible only if the system resolver is included and differs from others; (iii) legitimate CDN and geo-DNS behaviour will cause some false positives; (iv) resolver caching means a poisoned record may already have expired, or may not yet have appeared, when a check runs; (v) DNSSEC-based checks only apply to signed zones; and (vi) the public resolvers themselves are trusted as independent observers. An alert means *“investigate”*, never *“spoofing confirmed”*.

5. System Architecture and Design Methodology

5.1 System architecture

DNScope is a client–server application (Fig. 2). A browser dashboard sends the domain to a Python backend over HTTP. The backend's **DNS lookup service** acts as a stub resolver towards four recursive resolvers — the operating system's configured resolver (usually the ISP's or the campus network's) and three public anycast resolvers — and passes the responses to the analysis engine, which compares them with each other and with the domain's stored profile. Observations and alerts are kept in a local SQLite database. DNScope is an *active* measurement tool: it generates its own

queries and does not capture or inspect other traffic on the network.

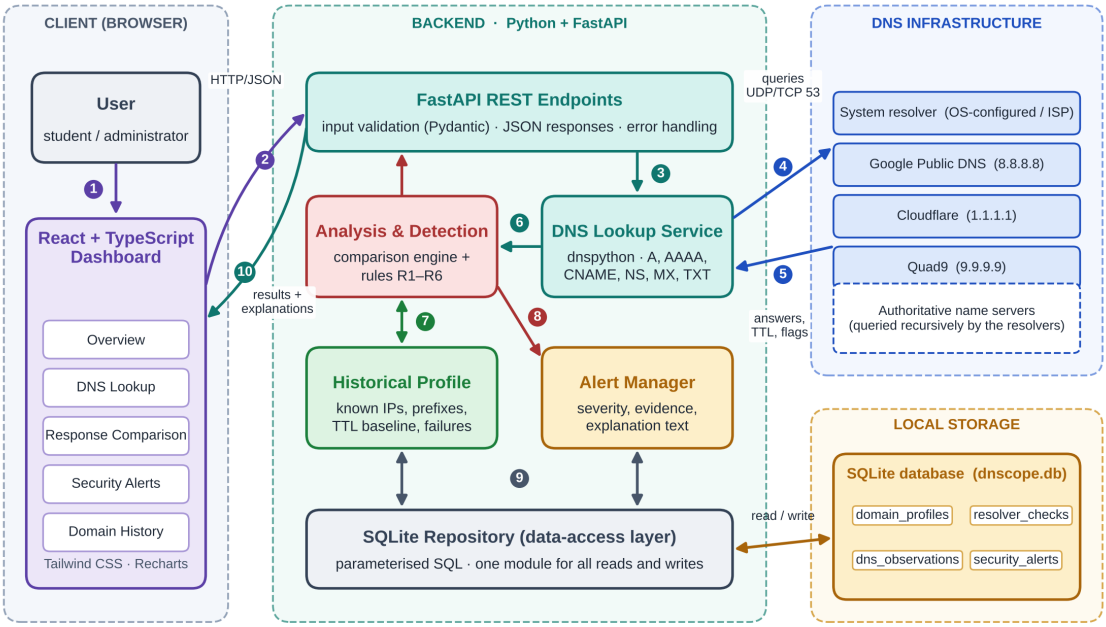


Fig. 2. DNScope system architecture. (1–3) the user's domain reaches the lookup service; (4–5) DNS queries and responses exchanged with four resolvers over UDP/TCP port 53; (6) parsed records pass to analysis; (7) comparison with the stored profile; (8) rule hits become alerts; (9) data stored; (10) results returned to the dashboard.

5.2 Measurement design: vantage points and query construction

Comparing answers is only meaningful if the observers are independent. DNScope uses the four resolvers in Table 4. The system resolver represents what the user's own machine actually receives, so it is the vantage point most exposed to on-path tampering or a poisoned local resolver; the public resolvers are operated by different organisations and reached over different network paths.

Table 4. Resolvers used as vantage points. Operator behaviour should be re-checked against provider documentation at implementation time.

Resolver	Address	Operator / network	DNSSEC validation	Notes for interpretation
System resolver	From OS configuration	ISP, campus or home router	Varies	What the user really gets; may rewrite NXDOMAIN or be on a tampered path
Google Public DNS	8.8.8.8	Google, anycast	Yes	Large shared cache; uses ECS with some CDNs
Cloudflare	1.1.1.1	Cloudflare, anycast	Yes	Does not send ECS, so CDN answers may differ from others
Quad9	9.9.9.9	Quad9 Foundation, anycast	Yes	Blocks known-malicious domains, which can create legitimate disagreement

For each resolver and record type, the lookup service builds a standard query with RD = 1 and an EDNS(0) OPT record with the DO bit set, so validating resolvers can report DNSSEC status through the AD flag [18], [3]. Queries are sent over UDP with a short timeout (planned default 2 s) and one retry; if a response arrives with TC = 1 the query is repeated over TCP [4]. For every response DNScope records the RCODE, the AA, TC, RA and AD flags, the answer RRset, each record's TTL, and the round-trip time. The **normaliser** then follows any CNAME chain to the final

A/AAAA records, lower-cases and sorts the RRset, groups addresses by /24 (IPv4) or /48 (IPv6) prefix, and keeps the *maximum* TTL seen across resolvers, because cached copies only ever count down from the authoritative value.

5.3 Detection workflow

Fig. 3 shows the workflow for one check. The domain is first validated (at most 253 characters, labels of 1–63 letters, digits or hyphens, internationalised names converted to their ASCII form). Queries are sent to all resolvers in parallel; timeouts, SERVFAIL and NXDOMAIN are stored as results rather than discarded, because failure patterns are themselves evidence. The normalised observations are compared across resolvers and against history, the rules in Table 5 are applied, and each triggered rule becomes an alert holding the evidence and a plain-language explanation that also lists possible legitimate causes.

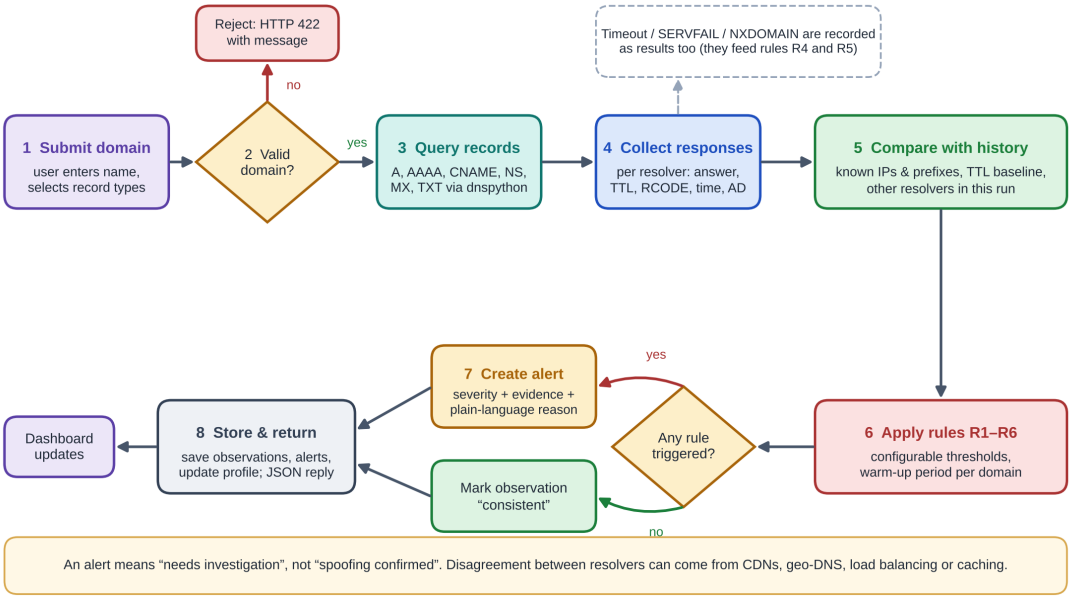


Fig. 3. DNS lookup and verification workflow, including input rejection, failure handling and the alert / no-alert branches.

Resolver disagreement alone does not establish spoofing: CDNs steer different resolvers to different edges, caches may hold an older answer after a legitimate change, and filtering resolvers alter answers on purpose. DNScope therefore raises severity only when independent signals agree — for example, a new address outside the domain's known prefixes, returned by only one resolver, with an unusually short TTL.

5.4 Detection rules

Table 5 defines the six rules. Thresholds are kept in one configuration file, and a per-domain **warm-up period** (planned default: 5 successful checks) suppresses the history-based rules R1 and R3 until a baseline exists.

Table 5. Rule-based detection rules. Severity levels are Info, Low, Medium and High; thresholds shown are planned defaults.

ID	Detection condition	Required input	Expected behaviour	Alert explanation (example)	Important limitations
R1	A/AAAA answer contains an address not in the profile's known set *and* outside all known /24 (IPv4) or /48 (IPv6) prefixes	Current answers; profile known_ips, known_prefixes; warm-up complete	Low alone; Medium if also R2 or R3	"New address 203.0.113.66 has never been seen for this domain and is outside its usual networks."	Migrations, new CDN edges and round-robin pools cause legitimate new addresses
R2	Within one run, at least one resolver's A/AAAA set shares no address and no prefix with the majority of other resolvers	Answers from at least 3 resolvers in the same run	Medium; High if the outlier is the system resolver and R1 also fires	"Quad9 returned addresses that no other resolver returned."	CDN and geo-DNS steering, filtering resolvers and stale caches all cause disagreement
R3	Maximum TTL observed in this run is below *ttl_low_ratio* × baseline (default 0.1) or above *ttl_high_ratio* × baseline (default 10)	Max TTL across resolvers; profile ttl_baseline (median of past max TTLs)	Info alone; raises severity of R1/R2 by one level	"TTL is 60 s, far below this domain's usual 300 s."	Operators lower TTLs before planned changes; cached TTLs count down (hence max, not mean)
R4	At least *fail_count* of the last *fail_window* checks (default 3 of 5) ended in timeout, SERVFAIL or REFUSED at all resolvers, or NXDOMAIN for a domain that previously resolved	RCODE/status per resolver; recent check history	Low; Medium for unexpected NXDOMAIN	"The domain failed to resolve in 3 of the last 5 checks."	Outages, expired domains and local network problems are far more common than attacks
R5	A validating resolver returns SERVFAIL, but the same query with the CD bit set returns an answer; or a domain previously seen with AD = 1 now returns AD = 0	RCODE and AD flag from validating resolvers; follow-up CD query; profile dnssec_seen	High for SERVFAIL-with-CD-answer; Medium for lost AD	"DNSSEC validation failed: the data could not be authenticated."	Applies only to signed zones; key-rollover mistakes by the operator also cause failures
R6	A public domain's A/AAAA answer contains a private, loopback or link-local address (e.g. RFC 1918 ranges [25], 127.0.0.0/8)	Answer addresses; Python ipaddress module	Medium; High if only one resolver returns it	"Answer 10.0.0.5 is a private address that is not reachable on the Internet."	Some organisations publish internal addresses on purpose (split-horizon DNS)

Threshold calibration. Defaults will be calibrated empirically. A *calibration set* of at least 30 popular domains, including CDN-heavy sites, will be checked repeatedly over several days with no injected attacks, so every alert on it is a false positive. Warm-up length, TTL ratios, the failure window and the prefix length used for "same network" will be adjusted until the false-positive rate is acceptable while the controlled attack scenarios still trigger. A separate *test set* is kept for evaluation so that tuning does not inflate results.

5.5 Threat coverage

Table 6 relates common DNS attacks to the network-level symptom a multi-resolver observer can see. It makes the limits of the approach explicit: attacks that change the authoritative data itself look the same from every vantage point.

Table 6. Mapping of DNS attacks and benign events to observable symptoms and rules.

Attack or event	Observable symptom	Rules	Detectable?
On-path tampering (rogue Wi-Fi, ARP spoofing on the LAN)	System resolver's answer differs from all public resolvers and from history	R2, R1	Yes, if the forged answer differs from the real one
Poisoned ISP / campus resolver cache	System resolver differs; often an unusual TTL	R2, R1, R3	Yes, while the poisoned entry is cached
Poisoned public resolver (one site of an anycast service)	One public resolver differs from the rest	R2	Partly; only the instance on DNScope's path is seen
Domain or registrar hijack; compromised authoritative server	All resolvers agree on a new address outside history	R1 (+R3)	Weakly; looks like a legitimate migration
Forged data for a DNSSEC-signed zone	SERVFAIL from validating resolvers; answer with CD = 1	R5	Yes, for signed zones
Redirect to internal hosts (e.g. DNS rebinding)	Private or loopback address in a public name's answer	R6	Yes
ISP NXDOMAIN rewriting (not an attack, but a redirect)	System resolver returns an address; others return NXDOMAIN	R2	Yes; flagged as Info with explanation
Legitimate CDN / geo-DNS / round-robin change	Different addresses in the same prefixes or operator	R1, R2 suppressed by prefix match	Should not alert (main false-positive risk)

5.6 Backend modules, dashboard and database

Fig. 4 shows the backend as a pipeline: an input validator, the DNS resolver service (dnspython [21]), the record normaliser, the comparison engine with its historical profile manager, the rule engine and an alert manager, with all storage going through a single repository module. The dashboard presents lookup results, a side-by-side resolver comparison with discrepancies highlighted, the alert list with evidence, and per-domain history including address changes and TTL trends.

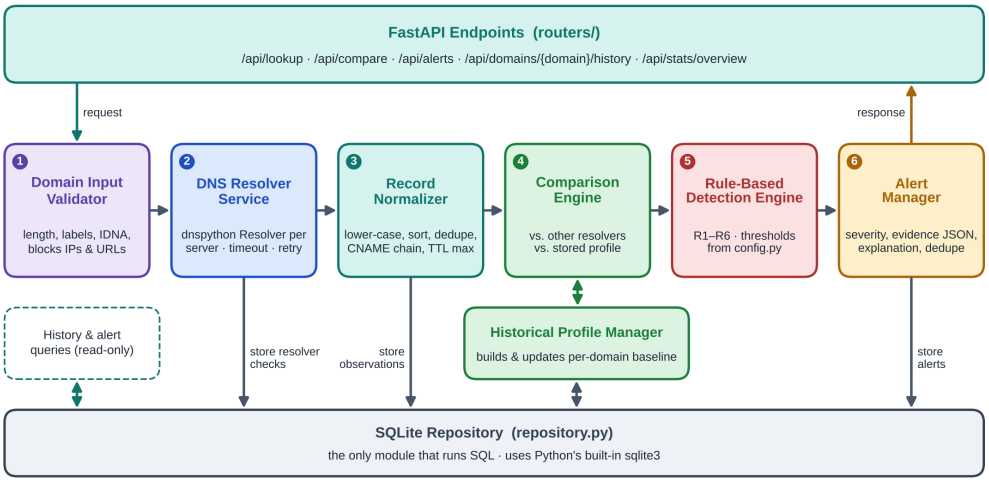


Fig. 4. Backend module architecture. The six pipeline modules run in order for every check; the Historical Profile Manager and SQLite Repository are shared services.

The database (Fig. 5) mirrors the measurement: **resolver_checks** stores one row per resolver per run (RCODE, status, round-trip time, AD flag); **dns_observations** stores each returned record with its type, value and TTL; **domain_profiles** holds the baseline used by the rules (known addresses and prefixes, TTL baseline, failure count, whether DNSSEC has been seen); and **security_alerts** stores each alert with an evidence field that cites the exact observations that triggered it.

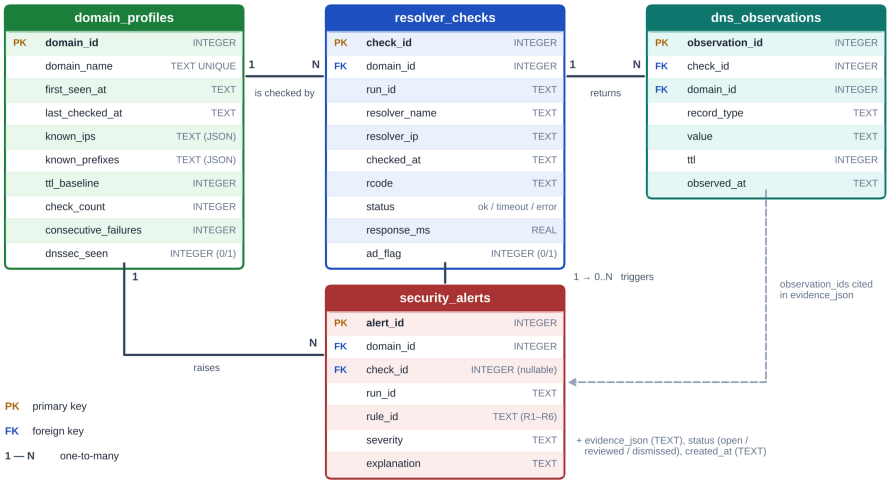


Fig. 5. Entity–relationship diagram of the DNScope database.

5.7 Testing and evaluation plan

Real DNS queries are used wherever possible. Attack conditions cannot ethically be created on real resolvers, so they are produced with a **mock resolver** that returns scripted DNS responses through the same interface; all such data is labelled *simulated*. Table 7 lists the planned scenarios.

Table 7. Planned test scenarios (“simulated” scenarios use the mock resolver).

Scenario	How it is produced	Expected outcome
Normal lookups	Real queries to stable domains, repeated over several days	No alerts after warm-up
Legitimate address changes	Real CDN-heavy and round-robin domains; simulated migration to a nearby prefix	No alert, or Low with a benign explanation
Resolver discrepancies	Real geo-DNS domains; mock resolver returning a different but plausible set	R2 with legitimate causes listed
Spoofed answer (simulated)	Mock resolver returns an address in an unrelated network with a short TTL, from one resolver only	R1 + R2 + R3 combine into a High alert
Private address (simulated)	Mock resolver returns 10.0.0.5 for a public domain	R6
DNSSEC valid / invalid	Real signed and unsigned zones; deliberately broken test zones if available; mock responses otherwise	No alert for valid zones; R5 for failures
Failures and truncation	Unreachable resolver; mock timeouts, SERVFAIL, NXDOMAIN; TC = 1 responses	TCP retry on TC; R4 after repeated failures

Each run is labelled in advance as *should alert* or *should not alert*. From true and false positives and negatives the evaluation will report **precision** = TP / (TP + FP), **recall** = TP / (TP + FN), **false-positive rate** = FP / (FP + TN) — most important for real CDN domains — and **detection latency** (median and 95th percentile time from query to alert, plus per-resolver round-trip time). Planned targets, to be revised after calibration, are recall of at least 0.9 on the simulated scenarios, a false-positive rate below 10% on real normal domains, and a median check time under 3 seconds. **These are targets, not results.** Because the simulated scenarios are designed with the rules in mind, recall on them will not be presented as real-world detection performance.

5.8 Expected outcomes

None of the following has been built yet. The expected outcomes are: a working prototype that queries four resolvers, parses and compares their responses and explains each alert; a dataset of real DNS observations (answers, TTLs, RCODEs, AD flags and round-trip times) for the calibration and test domains; an evaluation reporting the metrics above with an honest discussion of false positives on CDN domains and of attacks the method cannot see; and, for the student, practical understanding of DNS resolution, message formats, UDP/TCP transport, caching, anycast and DNSSEC.

6. References

[1] P. Mockapetris, “Domain names – concepts and facilities,” RFC 1034, IETF, Nov. 1987. doi: 10.17487/RFC1034.

[2] P. Mockapetris, “Domain names – implementation and specification,” RFC 1035, IETF, Nov. 1987. doi: 10.17487/RFC1035.

[3] J. Damas, M. Graff, and P. Vixie, “Extension mechanisms for DNS (EDNS(0)),” RFC 6891, IETF, Apr. 2013. doi: 10.17487/RFC6891.

[4] J. Dickinson, S. Dickinson, R. Bellis, A. Mankin, and D. Wessels, “DNS transport over TCP – implementation requirements,” RFC 7766, IETF, Mar. 2016. doi: 10.17487/RFC7766.

[5] M. Andrews, “Negative caching of DNS queries (DNS NCACHE),” RFC 2308, IETF, Mar. 1998. doi: 10.17487/RFC2308.

[6] C. Contavalli, W. van der Gaast, D. Lawrence, and W. Kumari, “Client subnet in DNS queries,” RFC 7871, IETF, May 2016. doi: 10.17487/RFC7871.

[7] J. Abley and K. Lindqvist, “Operation of anycast services,” RFC 4786, IETF, Dec. 2006. doi: 10.17487/RFC4786.

[8] V. Fuller and T. Li, “Classless inter-domain routing (CIDR): The Internet address assignment and aggregation plan,” RFC 4632, IETF, Aug. 2006. doi: 10.17487/RFC4632.

[9] Wireshark Foundation, “Wireshark User’s Guide.” [Online]. Available: https://www.wireshark.org/docs/wsug_html_chunked/

[10] The Zeek Project, “Zeek Documentation.” [Online]. Available: https://docs.zeek.org/

[11] Open Information Security Foundation, “Suricata User Guide.” [Online]. Available: https://docs.suricata.io/

[12] D. Atkins and R. Austein, “Threat analysis of the Domain Name System (DNS),” RFC 3833, IETF, Aug. 2004. doi: 10.17487/RFC3833.

[13] S. Son and V. Shmatikov, “The Hitchhiker’s Guide to DNS cache poisoning,” in *Security and Privacy in Communication Networks (SecureComm 2010)*, LNICST, vol. 50. Berlin, Germany: Springer, 2010, pp. 466–483. doi: 10.1007/978-3-642-16161-2_27.

[14] A. Hubert and R. van Mook, “Measures for making DNS more resilient against forged answers,” RFC 5452, IETF, Jan. 2009. doi: 10.17487/RFC5452.

[15] K. Man, Z. Qian, Z. Wang, X. Zheng, Y. Huang, and H. Duan, “DNS cache poisoning attack reloaded: Revolutions with side channels,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2020. doi: 10.1145/3372297.3417280.

[16] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, “DNS security introduction and requirements,” RFC 4033, IETF, Mar. 2005. doi: 10.17487/RFC4033.

[17] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, “Resource records for the DNS security extensions,” RFC 4034, IETF, Mar. 2005. doi: 10.17487/RFC4034.

[18] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, “Protocol modifications for the DNS security extensions,” RFC 4035, IETF, Mar. 2005. doi: 10.17487/RFC4035.

[19] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, and P. Hoffman, “Specification for DNS over Transport Layer Security (TLS),” RFC 7858, IETF, May 2016. doi: 10.17487/RFC7858.

[20] P. Hoffman and P. McManus, “DNS queries over HTTPS (DoH),” RFC 8484, IETF, Oct. 2018. doi: 10.17487/RFC8484.

[21] dnspython contributors, “dnspython documentation.” [Online]. Available: https://dnspython.readthedocs.io/

[22] M. Antonakakis, R. Perdisci, D. Dagon, W. Lee, and N. Feamster, “Building a dynamic reputation system for DNS,” in *Proc. 19th USENIX Security Symp.*, Washington, DC, USA, 2010. [Online]. Available: https://www.usenix.org/conference/usenixsecurity10/building-dynamic-reputation-system-dns

[23] L. Bilge, E. Kirda, C. Kruegel, and M. Balduzzi, “EXPOSURE: Finding malicious domains using passive DNS analysis,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2011. [Online]. Available: https://www.ndss-symposium.org/ndss2011/exposure-finding-malicious-domains-using-passive-dns-analysis

[24] D. Wendlandt, D. G. Andersen, and A. Perrig, “Perspectives: Improving SSH-style host authentication with multi-path probing,” in *Proc. USENIX Annual Technical Conf.*, Boston, MA, USA, 2008. [Online]. Available: https://www.usenix.org/conference/2008-usenix-annual-technical-conference/perspectives-improving-ssh-style-host

[25] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, “Address allocation for private internets,” RFC 1918, IETF, Feb. 1996. doi: 10.17487/RFC1918.